

MultiExplorer Application Architecture

Runtime component view - Windows desktop application, version 1.5.2

MultiExplorer - Runtime Application Architecture

Windows 10/11 x64 | C#.NET 8 WinForms | Version 1.5.2

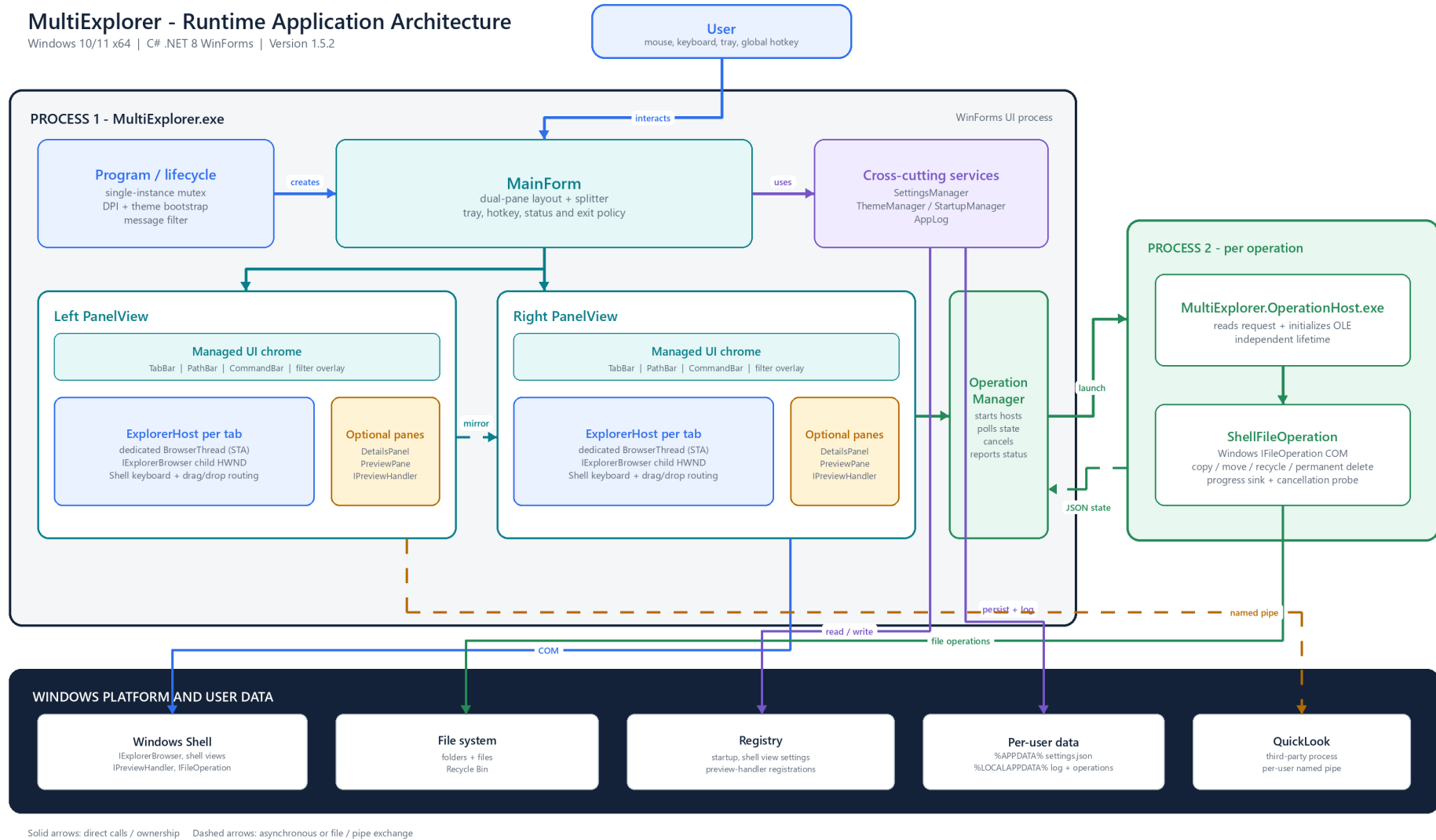


Diagram Notes

Scope and basis. Production runtime components, process and thread boundaries, Windows integrations, and per-user data flows. Build, test, and MSI packaging projects are outside this view. Prepared from README.md and the C# implementation for version 1.5.2, reviewed 11 September 2026.

1. Application bootstrap and lifetime

Program enforces a single instance with a named mutex. A later interactive launch broadcasts a registered Windows message so the existing window is restored. Startup launches remain silent when an instance already exists.

Settings and the selected theme are loaded before WinForms creates windows. The process opts into PerMonitorV2 DPI awareness, then creates MainForm on the UI STA thread.

2. Main window and dual-pane composition

MainForm owns the left and right PanelView controls, splitter behavior, tray integration, the configurable global show-window hotkey, status reporting, window persistence, and exit behavior while operations are active.

Each PanelView owns the tab, breadcrumb, command bar, type-to-filter overlay, details pane, preview pane, and a collection of ExplorerHost controls. Only the active tab host is visible. Mirror events navigate the opposite pane.

3. Native Explorer hosting

Each ExplorerHost launches a dedicated BrowserThread configured as an STA. That thread creates IExplorerBrowser in-process and parents its native child window beneath the WinForms control. Keeping shell modal loops off the UI thread prevents drag/drop activity from freezing the managed message pump.

Keyboard messages are routed to the current shell view, while application shortcuts and filtering keystrokes are intercepted where needed. Shell navigation events update cached paths without forcing synchronous cross-thread COM calls.

4. Selection, filtering, and preview

The details pane obtains icons and metadata from Windows Shell APIs. The preview pane resolves the registered handler CLSID from HKCR, creates the IPreviewHandler COM object, initializes it with the selected file or shell item, and hosts the preview in its own HWND.

Type-to-filter is a managed overlay. It enumerates the current directory asynchronously and presents matches in a WinForms ListView above the native shell view; it does not create a second Explorer window.

5. Isolated file-operation path

Copy, move, recycle-delete, and permanent-delete requests are serialized to per-operation JSON files under %LOCALAPPDATA%\MultiExplorer\Operations. OperationManager launches one OperationHost process for each request and polls state files every 500 ms.

OperationHost initializes OLE and invokes the Windows IFileOperation COM API. Its progress sink periodically publishes state. Cancellation is requested with a sentinel file and observed by a throttled probe. This file-based contract allows the UI and worker lifetimes to be monitored independently.

6. Persistence and platform integrations

SettingsManager stores application state in %APPDATA%\MultiExplorer\settings.json. AppLog writes a rolling log to %LOCALAPPDATA%\MultiExplorer\app.log. StartupManager maintains the current user's Run registry key. Some Explorer display preferences use the same per-user registry values as File Explorer.

When enabled, Space sends the selected path to a running QuickLook process over a SID-scoped named pipe. QuickLook is optional and remains outside the MultiExplorer process boundary.

Architecture Considerations

Concern	Implementation consequence
Responsiveness	The managed UI thread coordinates controls and application state, while each native Explorer view has a dedicated STA. Directory filtering also runs asynchronously with cancellation and debounce behavior.
Fault containment	Shell file operations run in a separate executable. The UI detects stale state and an exited host process, then marks the operation failed and reports it through the application log and status UI.
Windows fidelity	Navigation, thumbnails, context menus, drag/drop, preview handlers, and file operations intentionally delegate to Windows Shell COM interfaces. Behavior therefore follows installed shell extensions and per-user Explorer configuration.
Data locality	The application operates offline. Runtime state is local to the current Windows user: roaming AppData for settings and Local AppData for logs and operation coordination.
Theme and DPI	ThemeManager applies a shared managed palette and native dark-mode treatment. Shell views can be recreated after a live theme change. PerMonitorV2 DPI awareness aligns the host and embedded Explorer rendering contexts.
Optional dependency	QuickLook integration is feature-detected. If its process or named pipe is unavailable, the rest of the application continues to operate and the user receives a targeted message.

Boundary and interpretation notes

- Process 1 contains both managed controls and in-process Windows Shell COM objects. BrowserThread is a thread boundary, not another process.
- Process 2 is spawned per requested file operation. Communication is via files, not a named pipe or socket.
- The Windows Shell, registry, file system, and optional QuickLook process are external runtime dependencies, even though they execute on the same workstation.
- Installer, build scripts, and automated tests support delivery and verification but do not participate in the normal application runtime.

Maintenance note

Update this document when process boundaries, IPC contracts, storage locations, shell integration points, or the ownership of major UI components change. Routine visual or command additions usually require note updates only when they introduce a new dependency or responsibility boundary.